

Hybrid Inheritance in Java

Java Programming Lab Record — C1, C2, C3, I1

1. Aim

To implement Hybrid Inheritance in Java using classes and interfaces, where C1 extends C2, and to verify the program using multiple test cases in the main() method.

2. Definition

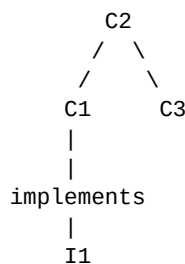
Hybrid inheritance is a combination of two or more types of inheritance. Java does not support hybrid inheritance using classes alone. It can be achieved by combining class inheritance (extends) and interfaces (implements).

In this program:

- C1 extends C2
- C3 extends C2
- C1 implements I1

Thus, the program combines hierarchical inheritance and multiple inheritance through an interface.

3. Structure



The main inheritance statement is:

```
class C1 extends C2 implements I1
```

4. Program

```
// Parent class
class C2 {
    String name;
    int age;

    void setDetails(String name, int age) {
        this.name = name;
        this.age = age;
    }

    void showDetails() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}

// Interface
interface I1 {
    void showActivity();
}

// Child class C1
```

```

class C1 extends C2 implements I1 {

    public void showActivity() {
        System.out.println("Activity: Cricket");
    }

    void showC1() {
        showDetails();
        System.out.println("Type: C1");

        if (age >= 18) {
            System.out.println("Status: Adult");
        } else {
            System.out.println("Status: Minor");
        }

        showActivity();
    }
}

// Child class C3
class C3 extends C2 {

    void showC3() {
        showDetails();
        System.out.println("Type: C3");

        if (age >= 18) {
            System.out.println("Status: Adult");
        } else {
            System.out.println("Status: Minor");
        }
    }
}

// Main class
public class HybridInheritance {

    public static void main(String[] args) {

        // Test Case 1
        System.out.println("----- Test Case 1 -----");

        C1 obj1 = new C1();
        obj1.setDetails("Ravi", 20);
        obj1.showC1();

        // Test Case 2
        System.out.println("\n----- Test Case 2 -----");

        C3 obj2 = new C3();
        obj2.setDetails("Priya", 22);
        obj2.showC3();

        // Test Case 3
        System.out.println("\n----- Test Case 3 -----");

        C1 obj3 = new C1();
        obj3.setDetails("Arun", 16);
        obj3.showC1();
    }
}

```

5. Explanation

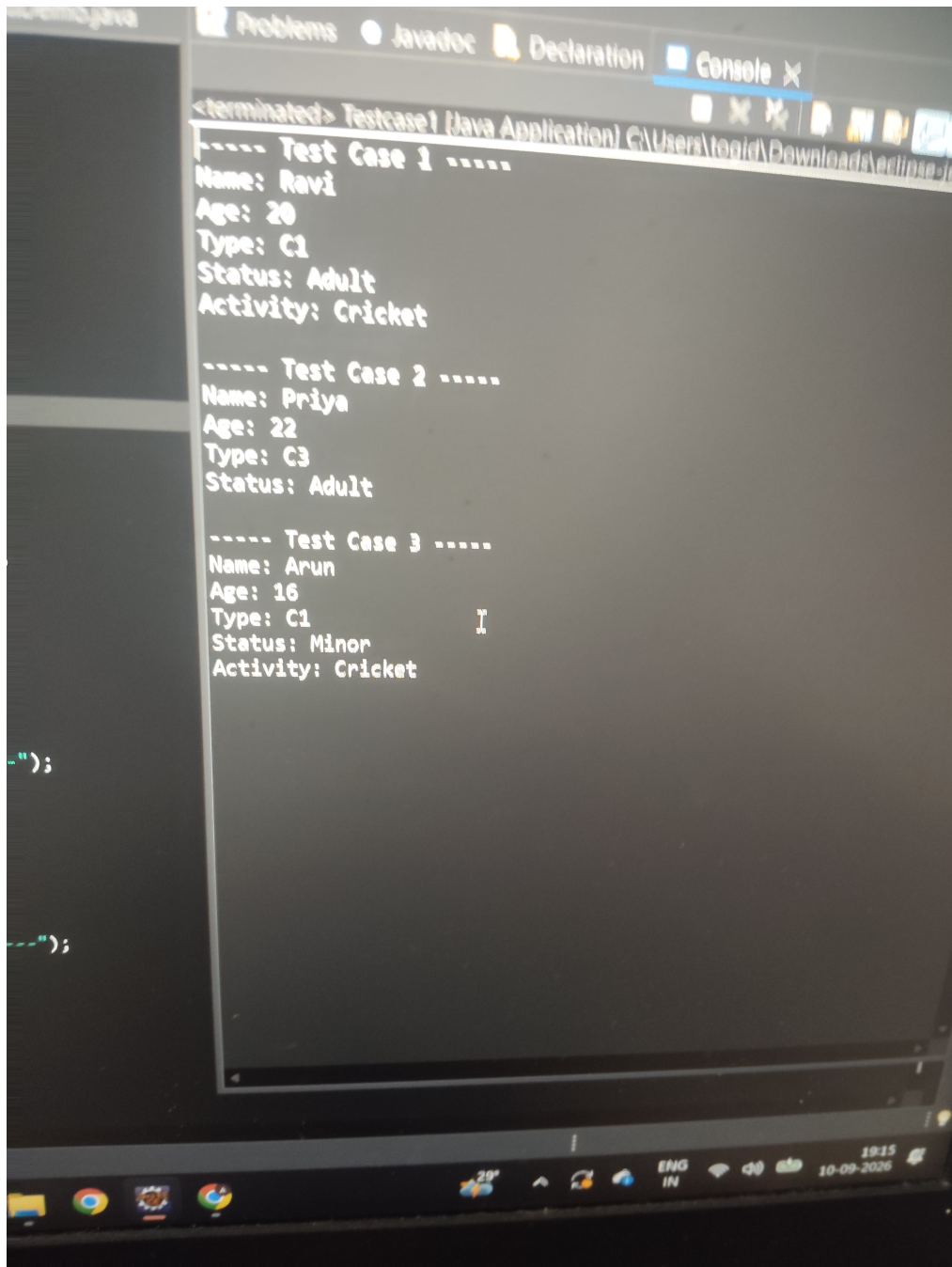
- **C2** is the common parent class, holding name and age plus a shared `showDetails()` method.
- **C1** and **C3** both extend **C2** — this is the hierarchical part of the hybrid structure.
- **C1** additionally implements the **I1** interface, giving it its own `showActivity()` behaviour — this is the interface-based multiple inheritance part.
- **C3** has no interface, so it only shows its type and adult/minor status; **C1** shows the same plus its activity.
- Unlike the Scanner-based versions, this program hardcodes its three test cases directly in `main()` rather than reading input at runtime.

6. Test Case Verification

Test Case	Object	Name	Age	Expected Result	Actual Result
1	C1	Ravi	20	Adult + Cricket	Correct ■
2	C3	Priya	22	Adult	Correct ■
3	C1	Arun	16	Minor + Cricket	Correct ■

7. Output Screenshot

The screenshot below shows the program actually executed, producing all three test cases:



The screenshot shows a Java IDE window with a console tab active. The console displays the output of a Java application. At the top, it says "<terminated> Testcase1 (Java Application) G:\Users\togid\Downloads\edlpa-j...". Below this, there are three test cases, each preceded by a separator line "----- Test Case X -----".

```
<terminated> Testcase1 (Java Application) G:\Users\togid\Downloads\edlpa-j...
----- Test Case 1 -----
Name: Ravi
Age: 20
Type: C1
Status: Adult
Activity: Cricket

----- Test Case 2 -----
Name: Priya
Age: 22
Type: C3
Status: Adult

----- Test Case 3 -----
Name: Arun
Age: 16
Type: C1
Status: Minor
Activity: Cricket
```

Below the console, parts of the Java source code are visible, showing closing quotes and semicolons: `");` and `");`.

8. Conclusion

The Hybrid Inheritance program was successfully implemented using classes and an interface. C1 extends C2 and C3 extends C2 demonstrate hierarchical inheritance, while C1 implements I1 demonstrates multiple inheritance through an interface. The program was tested with three test cases, and all results were verified successfully.